

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1107

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Madhan V

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I MUKESH M (192521368) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Short Answer Test

1. Suppose a software system is developed for a **Smart Hotel Management System** where guests can book rooms, cancel reservations, and make online payments.

(a) Explain the following object-oriented concepts:

- Class
- Object
- Inheritance
- Encapsulation
- Dynamic Binding

(b) Describe the following object relationships:

- Association
- Dependency
- Composition

(c) Explain how these concepts improve software maintainability and code reusability in a hotel management system.

answer :

1. CLASS

A class is a blueprint or template that defines the structure and behavior of objects. It specifies what data (attributes) an object will hold and what actions (methods) it can perform.

In Hotel System:

A Room class defines that every room has: room number, room type, price, availability status, and methods like `book_room()`, `cancel_reservation()`.

Class Room:

- Attributes: `room_id`, `room_type`, `price`, `availability`
- Methods: `book()`, `cancel()`, `update_price()`

2. OBJECT

An object is a specific instance created from a class. It's a real entity that exists in memory with actual values.

In Hotel System:

- Room 101 (Single room, 5000/night, Available) → This is ONE object
- Room 202 (Double room, 8000/night, Booked) → This is ANOTHER object

Both are created from the same Room class but have different values.

3. INHERITANCE

Inheritance allows a new class (child) to acquire properties and methods from an existing class (parent), promoting code reuse.

In Hotel System:

Parent Class: Room

- └─ Child Class: DeluxeRoom (inherits from Room)
- └─ Child Class: VIPRoom (inherits from Room)
- └─ Child Class: StandardRoom (inherits from Room)

Advantage: VIPRoom doesn't need to redefine `book_room()` or `cancel_reservation()` - it inherits these from Room class. It only adds special features like "Concierge Service" or "Free Airport Transfer".

4. ENCAPSULATION

Encapsulation is bundling data (attributes) and methods together and hiding internal details from the outside world. You control access through public methods.

In Hotel System:

Private: `room_price` (hidden)

Public Method: `get_price()` → Returns price safely

Public Method: `set_price(new_price)` → Updates price with validation

Why? If someone tries `room.price = -5000`, the system breaks. With encapsulation, `set_price()` validates: "Price must be positive."

5. DYNAMIC BINDING

Dynamic Binding means the same method call behaves differently based on which object calls it (determined at runtime, not compile time).

In Hotel System:

Method: `calculate_charge()`

For StandardRoom: `charge = price × nights`

For VIPRoom: `charge = (price × nights) + concierge_fee + breakfast`

For DeluxeRoom: `charge = (price × nights) + room_service_fee`

Same method name, different execution based on object type.

(b) Describe Object Relationships:

1. ASSOCIATION

Association is a relationship where two classes are connected but independent. If one is deleted, the other still exists.

In Hotel System:

Guest ↔ Room

A guest books a room. If guest cancels their account, the room still exists.

Type: One guest can book multiple rooms, one room can be booked by multiple guests (over time).

2. DEPENDENCY

Dependency means one class relies on another class to function, but they're loosely connected.

In Hotel System:

Payment ← Room

Payment class depends on Room class to know which room was booked and its price.

But Room doesn't depend on Payment.

3. COMPOSITION

Composition is a "part-of" relationship where one class is made up of other classes. If the parent is deleted, the parts are also deleted.

In Hotel System:

Room = Bed + Bathroom + Air Conditioner + Furniture

If Room is deleted, its Bed, Bathroom, AC are also deleted (they can't exist independently).

(c) How These Improve Maintainability and Reusability:

Concept	Benefit
Class & Object	Organize code logically; each class handles one responsibility
Inheritance	Write common code once in parent, reuse in children (DRY principle)
Encapsulation	Hide complexity; changes to internal logic don't affect other classes
Dynamic Binding	Add new room types without changing existing code
Relationships	Clear dependencies make code easier to understand and modify

Real Benefit: If you need to add a new room type (PentHouse), you just extend Room class. No need to rewrite booking, cancellation, or payment logic.

2. Consider developing a **Smart Education Management System**.

(a) Explain the following object-oriented design principles:

- Abstraction
- Encapsulation
- Information Hiding

(b) Describe the following SOLID principles:

- Single Responsibility Principle (SRP)
- Dependency Inversion Principle (DIP)

(c) Explain how these principles improve software scalability and maintainability.

answer :

(a) Explain OO Design Principles:

1. ABSTRACTION

Abstraction means showing only essential features and hiding unnecessary complexity.

In Education System:

Student Class:

- Public (Essential): name, roll_no, submit_assignment(), take_exam()
- Private (Hidden): internal_marks_calculation(), password_storage()

Teachers see submit_assignment() method but don't need to know HOW it's calculating grades internally.

2. ENCAPSULATION

Encapsulation hides data and provides controlled access through methods.

In Education System:

Bad : student.marks = 150 (Invalid - marks can't exceed 100)

Good : student.set_marks(85)

→ If marks > 100: reject

→ If marks valid: update

3. INFORMATION HIDING

Information Hiding means exposing only what's necessary; keep internal implementation secret.

In Education System:

User sees: "Your GPA is 8.5"

Hidden: How GPA is calculated (which formula, which semesters included)

If the college changes GPA calculation formula tomorrow, students don't need to know. Code remains same for users.

(b) Describe SOLID Principles:

1. SINGLE RESPONSIBILITY PRINCIPLE (SRP)

Each class should have ONE reason to change - meaning each class handles ONE specific task.

In Education System:

Bad Design:

Class Student:

- Store student data
- Calculate GPA
- Send emails
- Generate reports
- Manage attendance

If anything changes (email format, GPA calculation), Student class breaks.

Good Design:

Class Student → Only stores student data

Class GradesCalculator → Only calculates GPA

Class EmailService → Only sends emails

Class ReportGenerator → Only generates reports

Class AttendanceTracker → Only tracks attendance

Benefit: If email format changes, only EmailService needs modification.

2. DEPENDENCY INVERSION PRINCIPLE (DIP)

High-level modules should depend on abstractions, not low-level modules.

In Education System:

Bad - Tightly Coupled:

Class CourseScheduler:

```
def schedule():  
    email = EmailViaGmail() (Directly depends on Gmail)  
    sms = SMSViaAmazonSNS() (Directly depends on Amazon)
```

If you want to switch to EmailViaOutlook, entire code changes.

Good - Using Abstraction:

Interface: NotificationService

```
├── EmailViaGmail (implements NotificationService)  
├── EmailViaOutlook (implements NotificationService)  
└── SMSViaAmazonSNS (implements NotificationService)
```

Class CourseScheduler:

```
def schedule(notificationService: NotificationService):  
    notificationService.send() (Works with any service)
```

Benefit: Switch notification service without changing CourseScheduler code.

(c) How These Principles Improve Scalability & Maintainability:

Principle	Scalability	Maintainability
Abstraction	Easy to add new features without exposing complexity	Clear interfaces make code understandable
Encapsulation	Internal changes don't break external code	Data consistency guaranteed
SRP	Each class can be developed independently	Bugs isolated to one class
DIP	Plug-and-play components	Switching implementations is effortless

Real Example: Adding a new exam type (Practical Exam) requires minimal changes because each component (GradesCalculator, AttendanceTracker) is independent.

3. Suppose a software company is developing a **Smart Pharmacy Management System**.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- Feasibility Study
- Requirement Analysis
- System Design
- Implementation
- Testing
- Maintenance

(b) Describe the significance of each SDLC phase.

(c) Explain how SDLC helps reduce project risks and improve software quality.

answer :

(a) Explain SDLC Phases:

1. FEASIBILITY STUDY

Analyze whether the project is viable before starting development.

In Pharmacy System:

- Technical Feasibility: Can we build this with available technology? (Database, cloud servers, programming languages)
- Financial Feasibility: What's the budget? (₹5 lakhs, ₹10 lakhs?)
- Operational Feasibility: Will pharmacy staff easily use it?
- Legal Feasibility: Does it comply with pharmacy regulations?

Output: "Yes, proceed with project" or "No, project is not feasible"

2. REQUIREMENT ANALYSIS

Gather detailed requirements from pharmacy owners, staff, customers.

In Pharmacy System:

- Customers need: Online medicine ordering, delivery tracking, prescription upload
- Pharmacy staff need: Inventory management, expiry date alerts, billing
- Doctors need: Integration with prescription systems
- Management needs: Sales reports, profit analysis

Output: Detailed document listing ALL requirements (functional & non-functional)

3. SYSTEM DESIGN

Create architecture and design of the system before coding.

In Pharmacy System:

- Design Database: Tables for medicines, customers, orders, prescriptions
- Design User Interface: Login screen, medicine search, cart, checkout
- Design System Architecture: Frontend (Web/App), Backend (Server), Database
- Design Security: How to protect customer health data?

Output: Design documents, database schema, UI mockups, architecture diagrams

4. IMPLEMENTATION

Developers write actual code based on design.

In Pharmacy System:

- Write code for medicine database operations
- Write code for order processing
- Write code for payment integration
- Write code for delivery tracking

Output: Working software (not yet tested)

5. TESTING

Quality assurance team tests the software to find and report bugs.

In Pharmacy System:

- Unit Testing: Test each function individually (Does login work? Does cart calculation work?)
- Integration Testing: Test components together (Does order → payment → delivery work?)

- System Testing: Test entire system (Can customer complete order successfully?)
- UAT: Real users test (pharmacy staff, customers try actual scenarios)

Output: Bug reports, verified that system works as designed

6. MAINTENANCE

After deployment, fix issues and make updates.

In Pharmacy System:

- Bug Fixes: Customer reports payment failure → Fix immediately
- Updates: Add new payment method (UPI, Wallet)
- Performance Improvement: System slow? Optimize database queries
- Support: Help staff understand new features

Output: Stable, continuously improving system

(b) Significance of Each Phase:

Phase	Why Important
Feasibility	Prevents wasting money on impossible projects
Requirements	Ensures team builds what customer actually needs
Design	Poor design → Difficult to code, bugs, expensive fixes later
Implementation	Translates design into actual working system
Testing	Catches bugs BEFORE customer sees them
Maintenance	Keeps system running smoothly long-term

(c) How SDLC Reduces Risks & Improves Quality:

Without SDLC (Chaos):

Start coding → No clear requirements → Build wrong features → Customer unhappy → Expensive fixes → Project fails

With SDLC (Structured):

Feasibility → Requirements → Design → Code → Test → Deploy → Maintain → Happy customer ✓

Risk Reduction:

- Feasibility phase catches infeasible projects early

- Requirements phase prevents building wrong system
- Design phase catches architectural flaws before coding
- Testing phase finds bugs before customer uses it

Quality Improvement:

- Clear requirements = right features built
- Proper design = easier to code, maintain
- Thorough testing = reliable system
- Planned maintenance = stable long-term

4. Consider a project for developing a **Smart E-Commerce System**, where customer requirements frequently change due to market demands.

(a) Explain the following SDLC models:

- Waterfall Model
- V-Model
- Incremental Model
- Agile Model

(b) Compare the Waterfall Model and Incremental Model.

(c) Recommend the most suitable SDLC model for this project with justification.

answer :

1. WATERFALL MODEL

Sequential/Linear approach: Each phase must be completed before next begins. Like water flowing downward—no going backward.

Requirements → Design → Implementation → Testing → Deployment → Maintenance

↓

(Detailed)

↓

All requirements gathered FIRST

↓

Then entire design done

↓

Then entire coding done

↓

Then entire testing done

In E-Commerce System:

- Month 1: Gather ALL requirements (payment, shipping, inventory, search)
- Month 2: Complete ENTIRE system design
- Months 3-4: Code everything
- Month 5: Test everything
- Month 6: Deploy

Pros: Simple, clear timeline, good for well-defined projects

Cons: Cannot adapt to changing requirements, bugs discovered late cost more to fix

2. V-MODEL (Validation & Verification)

Testing is planned parallel to each development phase. For every development phase, there's a corresponding testing phase.

Requirements → Design → Implementation

↓ ↓ ↓

UAT → Integration Test → Unit Test

In E-Commerce System:

- While implementing payment module, test plan for payment is created
- While coding search feature, test cases for search are prepared
- Testing starts immediately, not at the end

Pros: Catches bugs early, better quality, systematic testing

Cons: Still rigid, expensive to change requirements mid-project

3. INCREMENTAL MODEL

Build system in small, manageable pieces (increments). Each increment adds new functionality.

Increment 1 (Sprint): Login + User Profile

↓ (Deploy & Test)

Increment 2 (Sprint): Search & Browse Products

↓ (Deploy & Test)

Increment 3 (Sprint): Shopping Cart

↓ (Deploy & Test)

Increment 4 (Sprint): Checkout & Payment

↓

Full System Ready

In E-Commerce System:

- Week 1-2: Release basic login system → Users can try it
- Week 3-4: Add product search → Users can browse
- Week 5-6: Add shopping cart → Users can add items
- Week 7-8: Add payment → Users can buy

Users get working system early; can give feedback for next increment.

Pros: Get working system early, can adapt to feedback, bugs caught early per increment

Cons: Requires good planning, integration can be complex

4. AGILE MODEL

Flexible, iterative development with frequent releases (every 1-2 weeks). Embrace changing requirements.

Sprint 1 (1-2 weeks): Login + Profile

↓ (Review & Retrospective)

Sprint 2 (1-2 weeks): Search & Browse

↓ (Review & Retrospective)

Sprint 3 (1-2 weeks): Shopping Cart

↓

Continuous improvement based on feedback

In E-Commerce System:

- Daily standup: "What did I do? What will I do? Any blockers?"
- Sprint review: Demo working features to customer
- Sprint retrospective: "What went well? What to improve?"
- Next sprint: Build based on feedback

Pros: Highly adaptive, customer satisfaction, frequent releases, motivated team

Cons: Requires disciplined team, documentation can be lacking, hard to predict timeline

(b) Compare Waterfall vs. Incremental:

Aspect	Waterfall	Incremental
Approach	Sequential, all at once	Small pieces, one-by-one
Requirements Change	Difficult to accommodate	Easier to incorporate
User Feedback	Only at the end	After each increment
Time to First Release	Long (6+ months)	Quick (2-3 weeks)
Testing	Entire system at end	Each increment tested
Cost of Changes	Very expensive if late	Less expensive
Risk	High (discover issues late)	Low (issues found early)
Best For	Fixed requirements	Evolving requirements

Example:

- Waterfall: Building a bridge—everything planned, no changes mid-way
- Incremental: Building a house—build kitchen first (use it), then bedroom, then bathroom

(c) Recommend Best SDLC Model for E-Commerce:

Best Model: AGILE

Justification:

1. Rapidly Changing Market: E-commerce needs to adapt fast to market trends
 - Competitor launches new feature → You add it in 2 weeks
 - Customer feedback says checkout is complex → Fix in next sprint
 - Market demands new payment method → Add in 1 week
2. Customer Involvement: Agile demands regular customer feedback
 - Weekly sprint reviews show progress
 - Customers see working features, suggest improvements
 - Happy customers = better product
3. Risk Mitigation: Build incrementally
 - Payment module tested in Sprint 2, not after 6 months
 - If bug found, only affects current increment
 - Major issues caught early
4. Competitive Advantage: Get to market fast
 - Waterfall: 6 months to launch basic system
 - Agile: 2 months to launch with core features, enhance later
5. Team Morale: See working product every 2 weeks
 - Motivation high
 - Quick wins boost confidence

Conclusion: E-commerce is competitive, customer-centric business. Agile's flexibility and speed make it ideal. Waterfall's rigidity would cause you to lose market opportunities.

5. Suppose a software analyst is designing a **Smart Hospital Information System**.

(a) Explain the role of UML in object-oriented software design.

(b) Describe the following UML diagrams:

- Use Case Diagram
- Sequence Diagram
- Communication Diagram
- Activity Diagram

(c) Explain how UML diagrams support software analysis, design, and documentation.

answer :

a) Explain Role of UML:

UML (Unified Modeling Language) is a standard way to visualize system design before coding. It helps communicate design ideas to team members, stakeholders, and developers.

Why UML Important:

- Blueprint: Like architectural drawings for buildings, UML shows software structure

- Communication: Non-technical stakeholders can understand system without code
- Documentation: Captures design for future reference
- Risk Reduction: Identify design flaws before expensive coding
- Standards: Everyone understands same notation

In Hospital System:

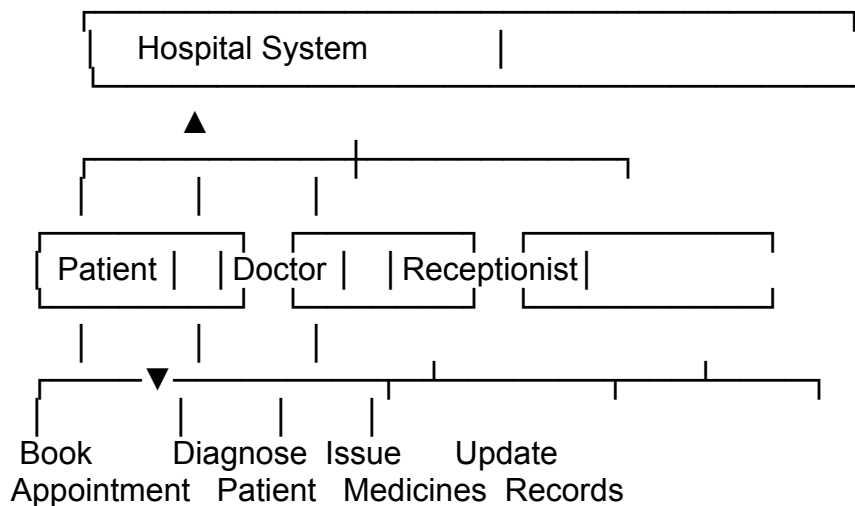
- Doctors don't know programming—UML diagrams show them how system works
- Developers use UML to understand what to code
- Project managers track development using UML artifacts

(b) Describe UML Diagrams:

1. USE CASE DIAGRAM

Shows what the system does from user's perspective. Identifies actors (users/external systems) and their interactions with the system.

In Hospital System:



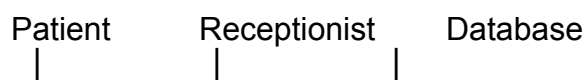
Actors: Patient, Doctor, Receptionist

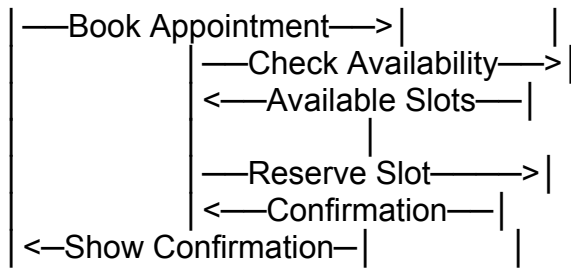
Use Cases: Book Appointment, Diagnose Patient, Issue Medicines, Update Records

2. SEQUENCE DIAGRAM

Shows how objects interact over time in specific scenarios. Shows order of messages/calls between objects.

In Hospital System - Scenario: Patient Books Appointment





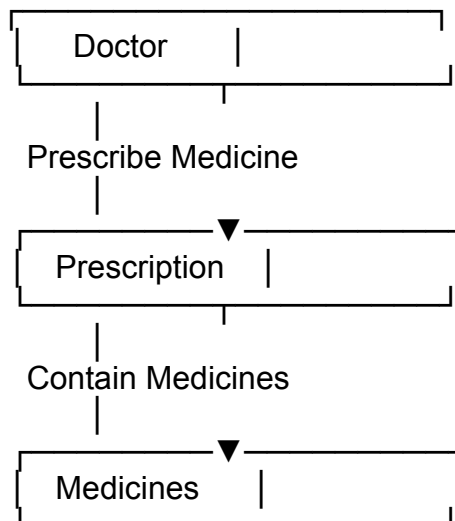
What it shows:

- Patient initiates booking
- Receptionist checks availability from Database
- Database returns available slots
- Receptionist reserves slot
- Database confirms
- Receptionist shows confirmation to Patient

3. COMMUNICATION DIAGRAM

Similar to Sequence diagram but shows relationships between objects more clearly. Emphasizes connections rather than time sequence.

In Hospital System - Scenario: Doctor Prescribes Medicine



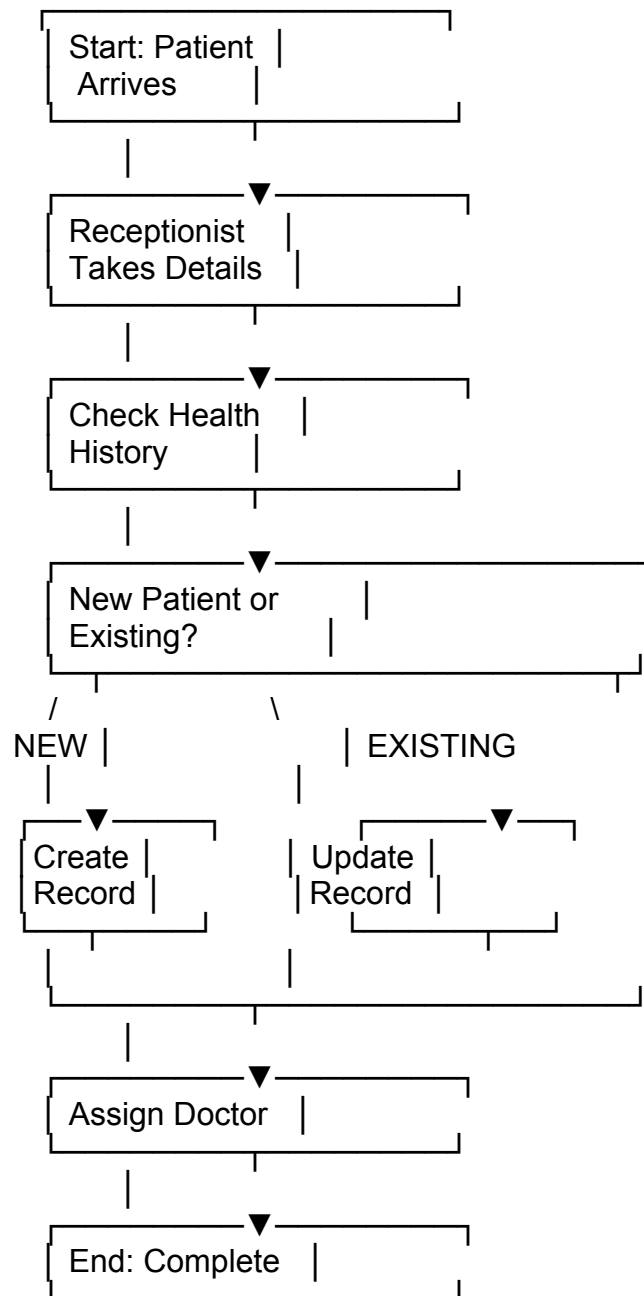
What it shows:

- Doctor prescribes medicine
- Prescription contains medicines
- Links show relationships between objects

4. ACTIVITY DIAGRAM

Shows business process flow - like a flowchart. Shows workflow, decision points, parallel activities.

In Hospital System - Patient Registration Flow:



What it shows:

- Process starts
- Receptionist takes details
- System checks if new or existing patient
- If new → Create record; if existing → Update record
- Then assign doctor
- Process ends

(c) How UML Diagrams Support Analysis, Design & Documentation:

ANALYSIS Phase (Understanding Requirements):

- Use Case Diagrams help identify ALL user needs
- Questions answered: "Who uses system? What do they do?"

- Example: Creates Use Case Diagram → Realizes we need "Emergency Patient Registration" (initially missed)

DESIGN Phase (Planning Solution):

- Sequence Diagrams show how different parts work together
- Communication Diagrams show relationships between components
- Activity Diagrams show data flow and decision logic
- Questions answered: "How do objects interact? What's the workflow?"
- Example: Sequence Diagram shows that Doctor-Patient interaction needs Authentication middleware

DOCUMENTATION Phase (Recording Design):

- UML diagrams ARE documentation
- New developers join → Can understand system from UML (no need for lengthy documents)
- Stakeholders review → Can see design without reading code
- Compliance & audits → UML proves system meets requirements

Real Example:

BEFORE UML:

Doctor: "System too slow when loading patient records"

Developer: "Let me check code... hmm, complex logic..."

Investigation takes 2 weeks

AFTER UML:

Doctor: "System slow in this flow" (points to Sequence Diagram)

Developer: "Ah! I see Database is called 3 times instead of 1"

Fix takes 2 hours

6. Consider developing a **Smart Railway Reservation System**.

- (a) Explain the concept of object-oriented software design.
- (b) Compare the structured software development approach with the object-oriented software development approach.
- (c) Explain how object-oriented design improves software flexibility, scalability, and reusability.

answer :

(a) Explain Object-Oriented Software Design:

Object-Oriented (OO) Design models software as collection of objects that have data and behavior, mirroring real-world entities.

In Railway System:

- Objects: Passenger, Train, Seat, Reservation, Ticket, Station
- Each object has:

- Data (State): Passenger has name, age, ID; Train has route, capacity, schedule
- Behavior (Methods): Passenger can bookTicket(), cancelReservation(); Train can departStation(), arriveStation()

Philosophy:

"Think in terms of THINGS (objects), not FUNCTIONS"

Example:

OO Thinking: "A Passenger object books a ticket"

Not Thinking: "A booking_function processes reservation_data"

(b) Compare Structured vs. Object-Oriented Approach:

STRUCTURED DEVELOPMENT:

Philosophy: Divide software into functions. Data and functions are separate.

Railway System (Structured):

Data:

- passenger_name[100]
- passenger_age[100]
- train_number[50]
- seat_availability[50]

Functions:

- input_passenger_data()
- validate_age()
- check_seat_availability()
- book_ticket()
- generate_ticket()
- cancel_reservation()

Structure: Functions process data; data is separate from functions.

OBJECT-ORIENTED DEVELOPMENT:

Philosophy: Bundle related data and functions as objects.

Railway System (OO):

Class Passenger:

- Data: name, age, ID
- Methods: bookTicket(), viewReservations(), cancelReservation()

Class Train:

- Data: trainNumber, route, capacity, schedule
- Methods: departStation(), arriveStation(), getAvailableSeats()

Class Seat:

- Data: seatNumber, type, availability, price
- Methods: reserveSeat(), cancelReservation()

Class Reservation:

- Data: reservationID, passenger, train, seats, date
- Methods: confirmReservation(), cancelReservation(), getTicket()

Each object is self-contained with both data and behavior.

Detailed Comparison Table:

Aspect	Structured	Object-Oriented
Data & Logic	Separate	Bundled together
Reusability	Functions can be reused, but	Objects are designed for reuse

	not designed for it	(inheritance, polymorphism)
Modification	Adding new feature = modify multiple functions + data structures	Adding new feature = extend existing class or create new class
Understanding	Need to trace through many functions to understand flow	Each object is self-contained, easier to grasp
Scalability	Becomes messy as system grows (spaghetti code)	Stays organized even as system grows
Testing	Test individual functions	Test objects in isolation and interaction
Real-World Mapping	Doesn't naturally map to real world	Maps directly to real-world entities (Passenger, Train)

Example Comparison: Adding New Feature "Loyalty Program"

Structured Approach (Difficult):

Current data structure:

- passenger_name[100]
- passenger_age[100]

Adding loyalty:

- passenger_loyalty_points[100] (Add new array)
- passenger_tier[100] (Add new array)

Existing functions break:

- input_passenger_data() (Need to input loyalty too)
- book_ticket() (Need to update loyalty points)
- generate_ticket() (Show loyalty tier on ticket)
- cancel_reservation() (Need to refund loyalty points)

IMPACT: Modify 4+ functions, high risk of bugs

Object-Oriented Approach (Easy):

Current:

Class Passenger:

- bookTicket()
- cancelReservation()

Adding loyalty:

Class LoyaltyProgram:

- points
- tier
- addPoints()
- redeemPoints()

Class Passenger (Updated):

- loyaltyProgram (new attribute) // Just reference loyalty object
- bookTicket() // Calls loyaltyProgram.addPoints()
- cancelReservation() // Calls loyaltyProgram.redeemPoints()

IMPACT: Add new class, minimal changes to existing code

(c) How OO Improves Flexibility, Scalability & Reusability:

1. FLEXIBILITY

Structured: Changing business rules requires rewriting functions

Current Rule: "One adult can book max 4 seats"

Change to: "VIP members can book 6 seats"

Structured: Modify `validate_seats()` function, test everything again

OO: Create `VIPPassenger` class inheriting from `Passenger`, override `booking_limit`

Result: OO allows changes through extension, not modification.

2. SCALABILITY

Structured: As system grows, code becomes difficult to manage

10,000 lines of interconnected functions

Hard to find where bug is

Changing one function may break 3 others

OO: Code organized in logical classes

Passenger class (500 lines)

Train class (400 lines)

Reservation class (600 lines)

Each class focused on one responsibility

Finding and fixing bugs is easy

Result: OO scales better; adding new features doesn't complicate entire codebase.

3. REUSABILITY

Structured: Functions are hard to reuse in new projects

`checkSeatAvailability()` function written for Railway System

Can't easily use in Bus Reservation System

(requires rewriting for different data structure)

OO: Classes can be reused across projects

Seat class (with reserveSeat, cancelReservation methods)

Can use in:

- Railway System
- Bus System (inherits Seat, adds busNumber)
- Movie Theater System (inherits Seat, adds screen_no)
- Hotel System (inherits Seat as Room)

No rewriting needed

Result: OO promotes code reuse; saves development time.

Real-World Impact for Railway System:

Requirement: Add new train type = "Premium Express"

Structured Approach:

1. Add new data fields: premium_fare_multiplier, priority_boarding
2. Modify check_seat_availability() function
3. Modify book_ticket() function
4. Modify generate_ticket() function
5. Modify cancel_reservation() function
6. Modify calculate_fare() function
7. Test 10+ scenarios

Time: 3-4 weeks

Risk: High (multiple functions modified)

Object-Oriented Approach:

Class PremiumExpress extends Train {

- premium_fare_multiplier = 1.5
- calculateFare() { return base_fare × 1.5 }

```
- priorityBoarding() { return true }  
}
```

Existing Passenger, Reservation, Seat classes work unchanged

Time: 2-3 days

Risk: Low (only new class created, existing classes untouched)

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weighta	Level 4 –	Level 3 –	Level 2 –	Level 1 –
----------	---------	-----------	-----------	-----------	-----------

	ge	Exemplary	Proficient	Developing	Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps